



CARRERA
LICENCIATURA EN CIENCIAS DE LA INFORMÁTICA

NOMBRE DE LA MATERIA
MINERÍA DE DATOS

TITULO DEL TRABAJO
DOCUMENTACIÓN TÉCNICA — PARTE II:
NORMALIZACIÓN DE LA BASE DE DATOS
COVID_19PRUEBAS

ALUMNO:
Tapia Reyes Diego
Gómez Fonseca Alexis

GRUPO: 7CM77
TURNO: Matutino

FECHA DE ENTREGA: 28 de Marzo del 2026



1. Descripción del Sistema Original

El archivo covid_19pruebas.sql es un dump de MySQL exportado con Navicat Premium el 03/03/2026. Representa un sistema de facturación y validación de atención COVID-19 del IMSS, que registra pacientes de otras instituciones (INSABI, SEDENA, SESA) atendidos en hospitales IMSS durante la pandemia.

Flujo del sistema

- Se registra el paciente en la tabla registro
- Se suben PDFs de Anexo 5 y 9 (documentación clínica)
- Se validan por CPIM y Finanzas en pacpdfval
- Los costos se calculan con base en la tabla tarifas

Tipo	Dump MySQL 5.1.41 exportado con Navicat Premium
Tamaño	~512 KB ~901,000 líneas
Tablas	17 en total
BD destino	covid_19pruebas

2. Esquema Completo de Tablas

2.1 Tablas de Catálogos / Lookup

Tabla	Campos principales y descripción
ooad	del (PK), delegoumae — Código delegación → nombre de estado
delegaciones	del, pwd13/14/15/16 — 40 delegaciones con contraseñas por año 2013-2016
admi	id (PK), nombre, ap_pat, ap_mat, usuario, pwd (texto plano)
unidades	id (PK), del, clues, nombre, clp, nivel (1=UMF/2=HGZ/3=UMAE), tipo, municipio (~24,677 registros)
cat_clues	id_cat (PK), clues, nom_imss, camas, jurisdicción, municipio, estado
catunimed13	clp (PK) — Catálogo 2013: dirección, director, capacidad, servicios (PQ1/PQ2/PQ3, lab, banco sangre, etc.)
tarifas	id (PK), cat, codigo, descripcion, t_20_1...t_23_3 — Tarifario COVID por semestre
tarifas_ant	Misma estructura que tarifas (versión anterior)



2.2 Tablas Operativas / de Pacientes

Tabla	Descripción y campos clave
registro (línea 171,737)	Cabecera: ~62,000 pacientes. id, nombre, curp, nss, dx, hospital origen/destino, 1 intervención por registro
registro_a (línea 234,540)	Detalle: ~583,000 intervenciones. id_a (PK), id (FK→registro). Repite TODOS los datos del paciente por cada intervención
registro_a_230323respaldo2	Respaldo de registro_a del 23/03/2023 — estructura idéntica
registro_b (línea 757,919)	Subconjunto simplificado de registro_a: ~4,815 registros, sin campos de costo/tarifa
registro_r (línea 757,967)	Snapshot de registro_a — sin campos de validación
pacpdfval (línea 16,970)	Validación: id (FK→registro), tinterv, timporte, vcpim, vfinanzas, fechas y observaciones
pdf (línea 61,754)	Archivos Anexo 5: del, id (FK), nombre de archivo PDF, fecsis, ipconfig
pdf9 (línea 117,596)	Archivos Anexo 9 — misma estructura que pdf
actualiza (línea 23)	Correcciones: id (FK→registro), nuevos, cantidad, IMPORTE

3. Problemas de Normalización Detectados

#	Problema	Descripción
1	Violación 3FN en registro_a	Todos los datos del paciente (nombre, curp, nss, hospitales...) se repiten en cada fila de intervención. Un paciente con 15 intervenciones tiene sus datos duplicados 15 veces.
2	Violación 1FN en tarifas	Las columnas t_20_1...t_23_3 son grupos repetitivos. Los periodos deben ser filas, no columnas.
3	Violación 4FN en catunimed13	PQ1, PQ2, PQ3 son dependencias multivaluadas. Los servicios (laboratorio, banco de sangre, hemodiálisis) también son independientes entre sí.
4	Hospitales embebidos como texto	hosp_origen, hosp_dest, estado, municipio se repiten en texto libre en lugar de referenciar a unidades.
5	Tablas redundantes/duplicadas	registro_a_230323respaldo2, registro_r, registro_b son subconjuntos de registro_a. tarifas_ant duplica tarifas.
6	Catálogos fragmentados	unidades, cat_clues y catunimed13 describen las mismas entidades médicas en tres tablas separadas.
7	Contraseñas en texto plano	admi.pwd y delegaciones.pwd13/14/15/16 almacenan contraseñas sin cifrar.



4. Normalización — Fase 0: Preparación

Siempre se trabaja sobre una base de datos nueva para no modificar los datos originales:

```
-- Crear base de datos nueva
CREATE DATABASE covid_normalizado CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE covid_normalizado;

-- Los INSERT usarán covid_19pruebas.tabla como fuente de datos
-- Ejemplo: INSERT INTO nueva_tabla SELECT ... FROM covid_19pruebas.tabla_original;
```

⚠ RECOMENDACIÓN: Antes de migrar datos, ejecuta estas verificaciones de calidad:

```
-- Registros con CURP nulo (afecta JOIN de paciente)
SELECT COUNT(*) FROM covid_19pruebas.registro WHERE curp IS NULL OR curp = '';

-- Formato real de fechas
SELECT DISTINCT fecha_derivacion FROM covid_19pruebas.registro LIMIT 20;

-- Inconsistencias en nombres de estados
SELECT DISTINCT estado FROM covid_19pruebas.registro ORDER BY estado;

-- Cuántas intervenciones no tienen tarifa asociada
SELECT COUNT(*) FROM covid_19pruebas.registro_a ra
LEFT JOIN covid_19pruebas.tarifas t ON t.codigo = ra.codigo
WHERE t.id IS NULL;
```

5. Primera Forma Normal (1FN)

Regla: Sin grupos repetitivos. Cada celda debe ser atómica.

5.1 Resolver columnas t_20_1...t_23_3 en tarifas

```
-- Tabla base sin columnas de período
CREATE TABLE tarifa (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  cat         VARCHAR(10),
  codigo      VARCHAR(10),
  esp_tron    TEXT,
  esp_deriv   TEXT,
  descripcion TEXT
);

-- Tabla de períodos (convierte 12 columnas en filas)
CREATE TABLE tarifa_periodo (
  id_tarifa INT,
  anio      YEAR,
  semestre TINYINT,
```



INSTITUTO POLITÉCNICO NACIONAL
UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y CIENCIAS
SOCIALES Y ADMINISTRATIVAS.
(U.P.I.I.C.S.A.)



```
    monto          INT,
    PRIMARY KEY (id_tarifa, anio, semestre),
    FOREIGN KEY (id_tarifa) REFERENCES tarifa(id)
);

-- Migrar datos base
INSERT INTO tarifa (id, cat, codigo, esp_tron, esp_deriv, descripcion)
SELECT id, cat, codigo, esp_tron, esp_deriv, descripcion
FROM covid_19pruebas.tarifas;

-- Unpivot de las 12 columnas de período
INSERT INTO tarifa_periodo (id_tarifa, anio, semestre, monto)
SELECT id, 2020, 1, t_20_1 FROM covid_19pruebas.tarifas WHERE t_20_1 IS NOT NULL
UNION ALL
SELECT id, 2020, 2, t_20_2 FROM covid_19pruebas.tarifas WHERE t_20_2 IS NOT NULL
UNION ALL
SELECT id, 2020, 3, t_20_3 FROM covid_19pruebas.tarifas WHERE t_20_3 IS NOT NULL
UNION ALL
SELECT id, 2021, 1, t_21_1 FROM covid_19pruebas.tarifas WHERE t_21_1 IS NOT NULL
UNION ALL
SELECT id, 2021, 2, t_21_2 FROM covid_19pruebas.tarifas WHERE t_21_2 IS NOT NULL
UNION ALL
SELECT id, 2021, 3, t_21_3 FROM covid_19pruebas.tarifas WHERE t_21_3 IS NOT NULL
UNION ALL
SELECT id, 2022, 1, t_22_1 FROM covid_19pruebas.tarifas WHERE t_22_1 IS NOT NULL
UNION ALL
SELECT id, 2022, 2, t_22_2 FROM covid_19pruebas.tarifas WHERE t_22_2 IS NOT NULL
UNION ALL
SELECT id, 2022, 3, t_22_3 FROM covid_19pruebas.tarifas WHERE t_22_3 IS NOT NULL
UNION ALL
SELECT id, 2023, 1, t_23_1 FROM covid_19pruebas.tarifas WHERE t_23_1 IS NOT NULL
UNION ALL
SELECT id, 2023, 2, t_23_2 FROM covid_19pruebas.tarifas WHERE t_23_2 IS NOT NULL
UNION ALL
SELECT id, 2023, 3, t_23_3 FROM covid_19pruebas.tarifas WHERE t_23_3 IS NOT NULL;
```

5.2 Resolver PQ1, PQ2, PQ3 en catunimed13

```
CREATE TABLE unidad_quirofano (
    clp          VARCHAR(30),
    numero_quirofano TINYINT,
    PRIMARY KEY (clp, numero_quirofano)
);

INSERT INTO unidad_quirofano (clp, numero_quirofano)
SELECT clp, 1 FROM covid_19pruebas.catunimed13 WHERE PQ1 IS NOT NULL AND PQ1 > 0
UNION ALL
SELECT clp, 2 FROM covid_19pruebas.catunimed13 WHERE PQ2 IS NOT NULL AND PQ2 > 0
UNION ALL
SELECT clp, 3 FROM covid_19pruebas.catunimed13 WHERE PQ3 IS NOT NULL AND PQ3 > 0;
```



6. Segunda Forma Normal (2FN)

Regla: Todo atributo no-clave debe depender de la clave primaria completa (aplica cuando la PK es compuesta).

✓ En este esquema la única tabla con PK compuesta después de 1FN es *tarifa_periodo*. El campo *monto* depende de (*id_tarifa*, *anio*, *semestre*) completo — está en 2FN correctamente.

Las demás tablas tienen PK de una sola columna, por lo que pasan automáticamente a 2FN. Las dependencias parciales detectadas son en realidad transitivas y se resuelven en la Fase 3.

7. Tercera Forma Normal (3FN)

Regla: Ningún atributo no-clave debe depender de otro atributo no-clave (sin dependencias transitivas).

⚠ Este es el paso más importante. El problema central es que *registro_a* repite los datos del paciente y del hospital en cada línea de intervención.

⚠ **RECOMENDACIÓN:** Orden de creación obligatorio por dependencias FK: *delegacion* → *estado* → *municipio* → *unidad_medica* → *paciente* → *tarifa* → *caso_covid* → *intervencion* → *documento* → *validacion*

7.1 Entidades geográficas/administrativas

```
CREATE TABLE delegacion (  
    del    VARCHAR(2) PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL  
);  
  
INSERT INTO delegacion (del, nombre)  
SELECT del, delegoumae FROM covid_19pruebas.ooad;  
  
-- -----  
  
CREATE TABLE estado (  
    id_estado INT PRIMARY KEY AUTO_INCREMENT,  
    nombre    VARCHAR(100) UNIQUE NOT NULL  
);  
  
INSERT INTO estado (nombre)  
SELECT DISTINCT estado FROM covid_19pruebas.registro  
WHERE estado IS NOT NULL AND estado != ''  
ORDER BY 1;
```



```
-- VERIFICAR antes de insertar: SELECT DISTINCT estado FROM covid_19pruebas.registro  
ORDER BY estado;
```

```
-- -----
```

```
CREATE TABLE municipio (  
    id_municipio INT PRIMARY KEY AUTO_INCREMENT,  
    id_estado    INT NOT NULL,  
    nombre       VARCHAR(100) NOT NULL,  
    FOREIGN KEY (id_estado) REFERENCES estado(id_estado)  
);
```

7.2 Catálogo unificado de unidades médicas

Las tres tablas unidades, cat_clues y catunimed13 describen las mismas entidades. Se usa unidades como base por ser la más completa:

```
CREATE TABLE unidad_medica (  
    clp          VARCHAR(20) PRIMARY KEY,  
    clues        VARCHAR(20),  
    nombre       VARCHAR(255),  
    del          VARCHAR(2),  
    nivel        TINYINT,          -- 1=UMF, 2=HGZ, 3=UMAE  
    tipo         VARCHAR(50),  
    numero       VARCHAR(20),  
    institucion  VARCHAR(200),  
    id_municipio INT,  
    FOREIGN KEY (del) REFERENCES delegacion(del),  
    FOREIGN KEY (id_municipio) REFERENCES municipio(id_municipio)  
);  
  
INSERT INTO unidad_medica (clp, clues, nombre, del, nivel, tipo, numero,  
institucion)  
SELECT clp, clues, unidad, del, nivel, tipo, numero, institucion  
FROM covid_19pruebas.unidades  
ON DUPLICATE KEY UPDATE clues = VALUES(clues);
```

7.3 Entidad Paciente

```
CREATE TABLE paciente (  
    id_paciente INT PRIMARY KEY AUTO_INCREMENT,  
    curp        VARCHAR(18),  
    nombre      VARCHAR(255),  
    appat       VARCHAR(255),  
    apmat       VARCHAR(255),  
    edad        INT,  
    genero      VARCHAR(10),  
    nss         VARCHAR(50),  
    dh          VARCHAR(100),    -- derechohabiencia  
    UNIQUE KEY uq_curp_nombre (curp, appat, apmat)  
);
```



```
-- Datos únicos desde registro (cabecera, no el detalle repetido)
INSERT INTO paciente (curp, nombre, appat, apmat, edad, genero, nss, dh)
SELECT DISTINCT curp, nombre, appat, apmat, edad, genero, nss, dh
FROM covid_19pruebas.registro
WHERE nombre IS NOT NULL;
```

7.4 Tabla Caso Covid

```
CREATE TABLE caso_covid (
    id INT PRIMARY KEY,
    id_paciente INT NOT NULL,
    no_fol VARCHAR(255),
    fecha_derivacion DATE, -- convertir desde VARCHAR
    fec_egreso DATE,
    dx VARCHAR(255),
    caso VARCHAR(50),
    ingreso VARCHAR(100),
    resclin LONGTEXT,
    marss VARCHAR(10),
    kccovid VARCHAR(10),
    tubo VARCHAR(10),
    fecha_inicio DATE,
    fecha_termino DATE,
    del VARCHAR(2),
    clp_origen VARCHAR(20),
    clp_destino VARCHAR(20),
    status INT,
    nuevos VARCHAR(100),
    FOREIGN KEY (id_paciente) REFERENCES paciente(id_paciente),
    FOREIGN KEY (del) REFERENCES delegacion(del),
    FOREIGN KEY (clp_origen) REFERENCES unidad_medica(clp),
    FOREIGN KEY (clp_destino) REFERENCES unidad_medica(clp)
);

INSERT INTO caso_covid (id, id_paciente, no_fol, fecha_derivacion, fec_egreso,
                        dx, caso, ingreso, resclin, del, clp_origen, clp_destino,
                        status)
SELECT r.id,
       p.id_paciente,
       r.no_fol,
       r.fecha_derivacion,
       r.fec_egreso,
       r.dx, r.caso, r.ingreso, r.resclin,
       r.del, r.clp, NULL, r.status
FROM covid_19pruebas.registro r
JOIN paciente p
ON r.curp = p.curp AND r.appat = p.appat AND r.apmat = p.apmat;
```

7.5 Tabla Intervención

```
CREATE TABLE intervencion (
```




```
id_intervencion INT PRIMARY KEY AUTO_INCREMENT,
id_caso         INT NOT NULL,
id_tarifa       INT,
codigo          VARCHAR(50),
anexo           VARCHAR(10),
especialidad    TEXT,
esp_deriv       TEXT,
descripcion     TEXT,
cantidad        INT,
tarifa_aplicada INT,
FOREIGN KEY (id_caso) REFERENCES caso_covid(id),
FOREIGN KEY (id_tarifa) REFERENCES tarifa(id)
);

INSERT INTO intervencion (id_caso, id_tarifa, codigo, anexo,
                          especialidad, esp_deriv, descripcion, cantidad,
                          tarifa_aplicada)
SELECT ra.id, t.id, ra.codigo, ra.anexo,
       ra.especialidad, ra.esp_deriv, ra.intervencion, ra.cantidad, ra.tarifa
FROM covid_19pruebas.registro_a ra
LEFT JOIN tarifa t ON t.codigo = ra.codigo;
```

7.6 Tabla Documento (resuelve pdf1/pdf2/pdf3)

```
CREATE TABLE documento (
  id_documento INT PRIMARY KEY AUTO_INCREMENT,
  id_caso      INT NOT NULL,
  tipo_anexo   VARCHAR(5),      -- '5' o '9'
  nombre_archivo TEXT,
  fecsis       VARCHAR(50),
  ipconfig     VARCHAR(50),
  FOREIGN KEY (id_caso) REFERENCES caso_covid(id)
);

INSERT INTO documento (id_caso, tipo_anexo, nombre_archivo, fecsis, ipconfig)
SELECT id, '5', pdf, fecsis, ipconfig FROM covid_19pruebas.pdf WHERE id IS NOT NULL;

INSERT INTO documento (id_caso, tipo_anexo, nombre_archivo, fecsis, ipconfig)
SELECT id, '9', pdf, fecsis, ipconfig FROM covid_19pruebas.pdf9 WHERE id IS NOT
NULL;
```

7.7 Tabla Validación

```
CREATE TABLE validacion (
  id_caso INT PRIMARY KEY,
  cpimvalidador VARCHAR(10),
  vcpim        VARCHAR(10),
  cpimfecha    VARCHAR(50),
  ocpim        TEXT,
  finanzasvalidador VARCHAR(10),
  vfinanzas     VARCHAR(10),
  finanzasfecha VARCHAR(50),
```



```
ofinanzas          TEXT,
tinterv            DECIMAL(32,0),
timporte           DECIMAL(42,0),
FOREIGN KEY (id_caso) REFERENCES caso_covid(id)
);

INSERT INTO validacion
SELECT id, cpimvalidador, vcpim, cpimfecha, ocpim,
       finanzasvalidador, vfinanzas, finanzasfecha, ofinanzas,
       tinterv, timporte
FROM covid_19pruebas.pacpdfval;
```

8. BCNF y Cuarta Forma Normal (4FN)

Regla 4FN: Una tabla no debe tener dos o más dependencias multivaluadas independientes entre sí.

8.1 Servicios de unidades médicas (catunimed13)

Una unidad médica puede tener múltiples servicios de forma independiente (laboratorio, banco de sangre, hemodiálisis). Guardarlos juntos genera tuplas falsas al hacer JOIN:

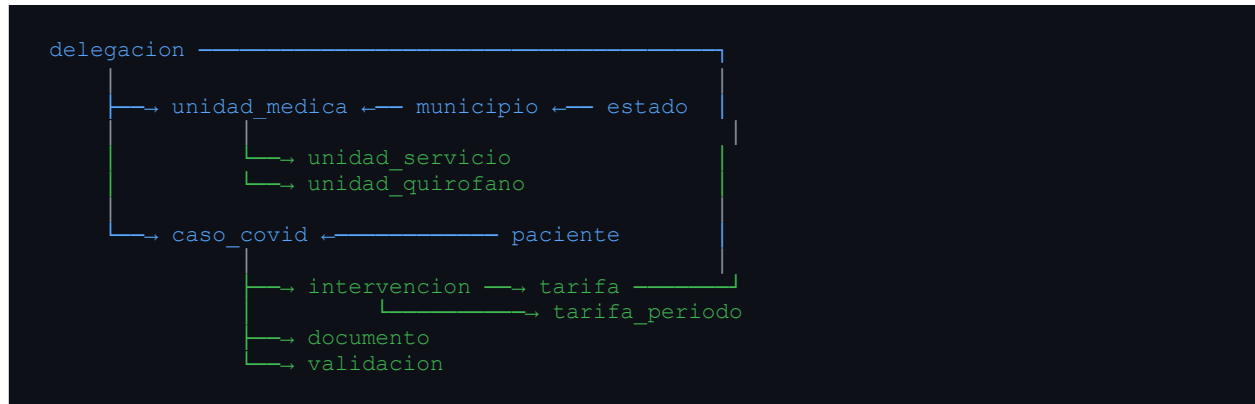
```
CREATE TABLE unidad_servicio (
  clp          VARCHAR(30),
  tipo_servicio VARCHAR(50), -- 'laboratorio', 'banco_sangre', 'hemodialisis',
  etc.
  proveedor    VARCHAR(100),
  disponible    TINYINT(1),
  PRIMARY KEY (clp, tipo_servicio),
  FOREIGN KEY (clp) REFERENCES unidad_medica(clp)
);

INSERT INTO unidad_servicio (clp, tipo_servicio, proveedor, disponible)
SELECT clp, 'laboratorio', prov_lab, laboratorio FROM
covid_19pruebas.catunimed13 UNION ALL
SELECT clp, 'banco_sangre', prov_bco, banco_sangre FROM
covid_19pruebas.catunimed13 UNION ALL
SELECT clp, 'hemodialisis', prov_hem, hemo_int FROM
covid_19pruebas.catunimed13 UNION ALL
SELECT clp, 'min_invasion', prov_min, minima_invasion FROM
covid_19pruebas.catunimed13;
```

8.2 Relaciones caso-intervención y caso-documento

✓ Un caso tiene múltiples intervenciones Y múltiples documentos de forma independiente. Al tenerlos en tablas separadas (Fase 3) se cumple automáticamente 4FN.

9. Diagrama del Esquema Normalizado



10. Tablas que Pueden Descartarse

Tabla original	Motivo para descartar
registro_a_230323respaldo2	Backup — datos ya contenidos en intervencion
registro_b	Subconjunto simplificado de registro_a — redundante
registro_r	Snapshot/vista de registro_a — redundante
tarifas_ant	Va como filas adicionales en tarifa_periodo con años anteriores
pdf, pdf9	Consolidadas en la tabla documento
actualiza	Sus correcciones van en intervencion con los importes corregidos

11. Recomendaciones antes de Ejecutar

#	Recomendación
1	Respetar el orden de creación de tablas por dependencias FK: delegacion → estado → municipio → unidad_medica → paciente → tarifa → caso_covid → intervencion → documento → validacion
2	Verificar CURPs nulos antes de migrar pacientes: SELECT COUNT(*) FROM covid_19pruebas.registro WHERE curp IS NULL OR curp = "";
3	Revisar formato real de fechas (pueden ser VARCHAR con formatos mixtos): SELECT DISTINCT fecha_derivacion FROM covid_19pruebas.registro LIMIT 20;



4	Limpiar inconsistencias de texto en estados/municipios (mayúsculas, acentos, abreviaciones) antes de insertar en la tabla estado.
5	Verificar cuántas intervenciones no tienen tarifa: el LEFT JOIN en el INSERT de intervencion puede dejar muchos registros sin id_tarifa.
6	El campo clp_origen en caso_covid quedó como NULL en el INSERT inicial. Completarlo cruzando con cat_clues usando el campo clues de registro.
7	Las contraseñas en admi.pwd deben hashearse en producción. En el entorno académico documentar como problema de seguridad detectado.
8	Ejecutar queries de verificación después de cada fase para confirmar conteos: SELECT COUNT(*) FROM cada nueva tabla vs la original.

12. Queries de Verificación

Ejecutar después de cada migración para confirmar que los datos quedaron correctos:

```
USE covid_normalizado;

-- Contar registros en cada tabla normalizada
SELECT 'delegacion'      , COUNT(*) FROM delegacion      UNION ALL
SELECT 'estado'          , COUNT(*) FROM estado          UNION ALL
SELECT 'unidad_medica'   , COUNT(*) FROM unidad_medica   UNION ALL
SELECT 'paciente'        , COUNT(*) FROM paciente        UNION ALL
SELECT 'tarifa'           , COUNT(*) FROM tarifa          UNION ALL
SELECT 'tarifa_periodo'  , COUNT(*) FROM tarifa_periodo  UNION ALL
SELECT 'caso_covid'      , COUNT(*) FROM caso_covid      UNION ALL
SELECT 'intervencion'    , COUNT(*) FROM intervencion    UNION ALL
SELECT 'documento'       , COUNT(*) FROM documento       UNION ALL
SELECT 'validacion'      , COUNT(*) FROM validacion;

-- Verificar que caso_covid tiene los mismos IDs que registro original
SELECT COUNT(*) FROM caso_covid; -- debe ser igual a registro original

-- Verificar intervenciones sin tarifa
SELECT COUNT(*) FROM intervencion WHERE id_tarifa IS NULL;

-- Verificar periodos migrados correctamente
SELECT anio, semestre, COUNT(*) FROM tarifa_periodo GROUP BY anio, semestre ORDER BY
anio, semestre;
```